Vellore Institute of Technology, Vellore

[School name]

Computer Networks Lab ([Course code])

Review 3 – Final Research Project Report

When Does Adapting Concurrency and Range Size Help?

A Controlled Study of Parallel HTTP Range Downloads

Submitted by

Name	Registration number
Sourajyoti Basak	24BCE0774
Shwetank Shrijan	24BDS0141
Ayush Konar	24BDS0107

Under the guidance of

[Faculty guide name]

October 2026

Contents

Abstract **1**

1 Complete Working Project **2**

2 Final System Architecture **3**

3 Complete Implementation **5**

3.1 Key Algorithms 5

4 Live Demonstration **8**

5 Test Results **10**

5.1 Reliability 10

5.2 Goodput 11

6 Performance Analysis **12**

6.1 Experimental Setup 12

6.2 Engine Ablation (230 runs) 14

6.3 Does Adaptation Help? 15

6.4 Adaptation Speed 15

6.5 Online Concurrency Tuner 16

6.6 Other Searches for an Adaptation Opportunity 17

6.7 Real Networks 17

6.8 Limitations 18

7 Comparison with Existing System **20**

7.1 What the Comparison Shows 20

8 Achievement of Objectives **21**

8.1 Deviations from the Review 1 Plan 22

8.2 Use of Prior Work 23

9 Innovation and Real-World Application **24**

10 Future Enhancement **25**

11 Outcome **26**

References **27**

List of Figures

2.1 Final system architecture. Only the Fetcher talks to the network; policies act only through the controllers. . . . 3
2.2 UML class diagram of the final implementation. Every policy, rule-based, learned or online-tuned, implements one interface. . . . 4
2.3 Level-1 data flow. Each attempt's result reaches the policy, which can change *N* and *S* for later allocations. . . . 4

3.1 Worker algorithm. . . . 6

4.1 Live download of `linux-3.0.tar.xz` (63.8 MB) from cdn.kernel.org with 8 connections; the SHA-256 matches the published value. . . . 8
4.2 Benchmark log on the paced test server: ours about 20 s, aria2 about 54 s. . . . 8
4.3 The automated Go test suite passing. . . . 9

5.1 Extra bytes re-downloaded under the original 30 s whole-request timeout (capped server, 32 MiB ranges). . . . 11

6.1 Reference pacing schedule D2. A request that starts in the first 4 s keeps its slow rate until it finishes. . . . 13
6.2 Completion time before and after the engine fix, and aria2, in four scenarios (10 blocks each). . . . 14
6.3 Adaptive controllers against the best stationary setting. . . . 15
6.4 Online concurrency tuner against fixed settings and aria2 (8 blocks per scenario). . . . 16
6.5 Live-internet pilot: how much slower one global fixed setting and aria2 were than the per-condition best. . . . 18

List of Tables

3.1 Implemented modules 5

5.1 Verification summary 10
5.2 Test cases for the final additions 10
5.3 Goodput in MB/s (geometric mean of 10 runs; fixed engine) 11

6.1 Platform 12
6.2 Server scenarios 12
6.3 Engine-ablation arms. “T30”: original 30 s whole-request deadline, no splitting. “fixed”: no deadline, splitting on. 14
6.4 Adaptation speed: median seconds, with the range over runs 16
6.5 Online tuner results; ratios are tuner / other 16
6.6 Studies that looked for a benefit from adaptation (“Testbed”: local server and emulated network) 17
6.7 Learned-policy evaluations (ratio $>$ 1: slower than the comparison) 18

7.1 Ours (best fixed setting, fixed engine) against aria2 20

8.1 Review 1 objectives and outcomes 21
8.2 Deviations and their reasons 22
8.3 Techniques from the Review 1 literature survey and how this project used them 23

11.1 Publication plan 26

Abstract

Segmented download managers fetch a file as byte ranges over parallel HTTP connections. Their speed depends on two settings: the number of connections, N, and the range size, S. This research project asks when changing N and S during a download actually helps. To answer it, we built an instrumented Go downloader whose N and S a policy can change mid-download, a reproducible test platform around it, and a controlled evaluation against aria2. The policies compared were fixed settings, rule-based controllers, a contextual bandit (LinUCB), deep reinforcement learning (DQN) and an online concurrency tuner in the style of prior work.

The main finding is that **a well-chosen fixed setting was nearly always enough**. On a server with per-response rate pacing, a fixed 16×4 MiB setting took 62.4 % less time than aria2 (time ratio 0.376 over 30 paired runs), a gap we trace mainly to request timing rather than a general advantage. On public mirrors, one fixed setting was within 3.7 % [1.0, 4.8] of the best per-condition choice. In recorded cellular traces, a clairvoyant oracle found no exploitable concurrency headroom. Rule-based, selector, learned and online-tuned controllers did not beat the best fixed setting in the comparisons we ran. A 230-run engine ablation showed that aria2's earlier lead on a concurrency-capped server came from two engine differences, a whole-request timeout and the lack of straggler splitting. Once both were fixed, our downloader matched aria2 or came within 1.6 % of it, and the best fixed setting beat the causal controller, the workload selector and the online tuner. No learned policy outperformed the baselines in the comparisons we ran.

1. Complete Working Project

This is a research project. Its question is when changing the number of connections N and the range size S during a download actually helps. Answering it needed a working experimental system, which has four parts, all working and tested:

1. **Downloader.** A Go command-line tool that downloads a single file from one server over HTTP/1.1 Range requests (resume, authentication and HTTP/2 are out of scope). It probes the server for range support and file size, fetches ranges over N parallel HTTP/1.1 connections, writes them at their offsets, and checks the result against an expected SHA-256.
2. **Control layer.** Concurrency and range-size controllers driven by a common policy interface. Implemented policies: fixed; throughput rules for N, S and both; a workload selector; a causal drop controller; an online concurrency tuner; LinUCB; and DQN agents.
3. **Test platform.** A local HTTP server with configurable per-response pacing, a soft concurrency cap and fault injection; Linux `tc netem` for delay and loss; Mahimahi replay of recorded cellular traces; and a runner for live downloads from public mirrors.
4. **Evidence pipeline.** Every run writes a JSON record. Each protocol was written down before its first run, with later amendments dated and disclosed. A claims checker recomputes all 144 numbers in the accompanying research-paper draft from the raw records; the numbers in this report come from the same analysis files.

2. Final System Architecture

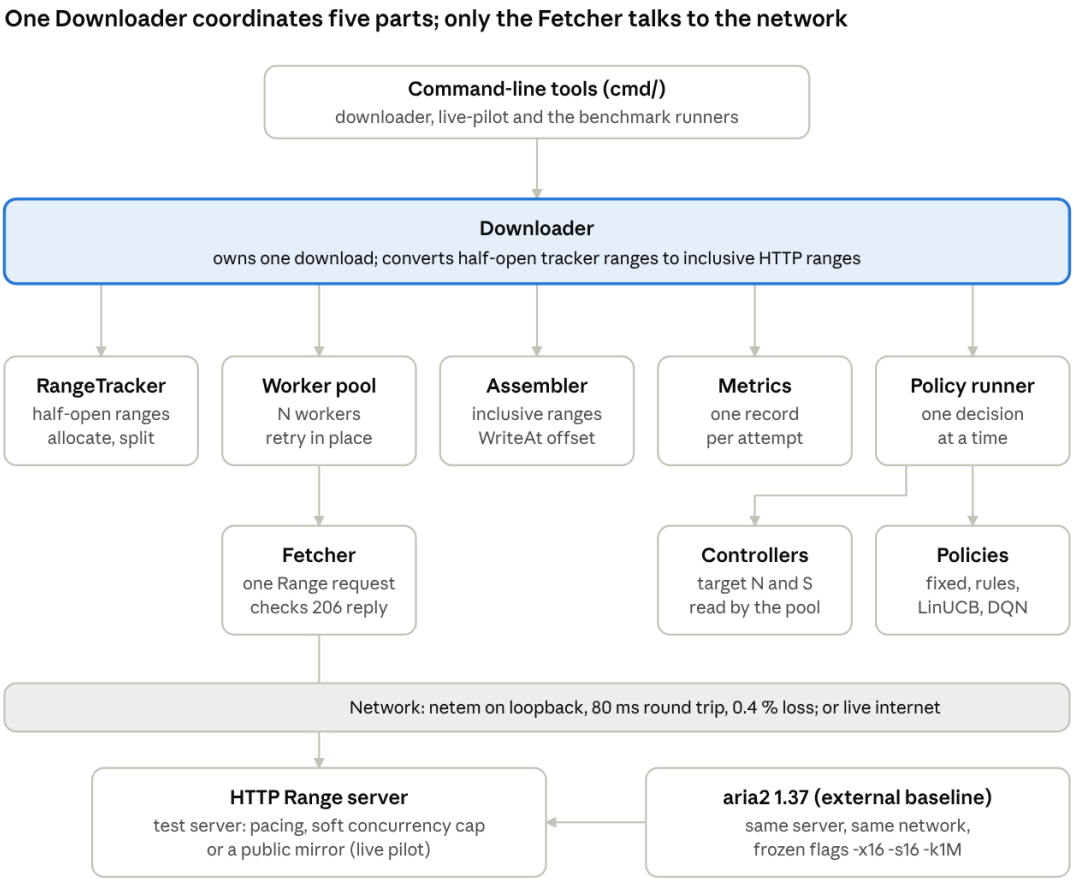


Figure 2.1: Final system architecture. Only the Fetcher talks to the network; policies act only through the controllers.

Beyond the core engine described in Review 2, the final architecture includes three parts used in the final evaluation:

- **Straggler splitting (opt-in).** When no new range is left, an idle worker takes over the second half of the unreceived part of the in-flight range with the most bytes still to come, as aria2 does. It is off by default, so every earlier result is unchanged.
- **Online concurrency tuner.** A new policy in `internal/policy` that tunes N from measured aggregate throughput (Section 6.5).
- **Probe and integrity check.** The CLI checks that the server answers a one-byte Range request with `206` before downloading, and verifies an optional SHA-256 afterwards.

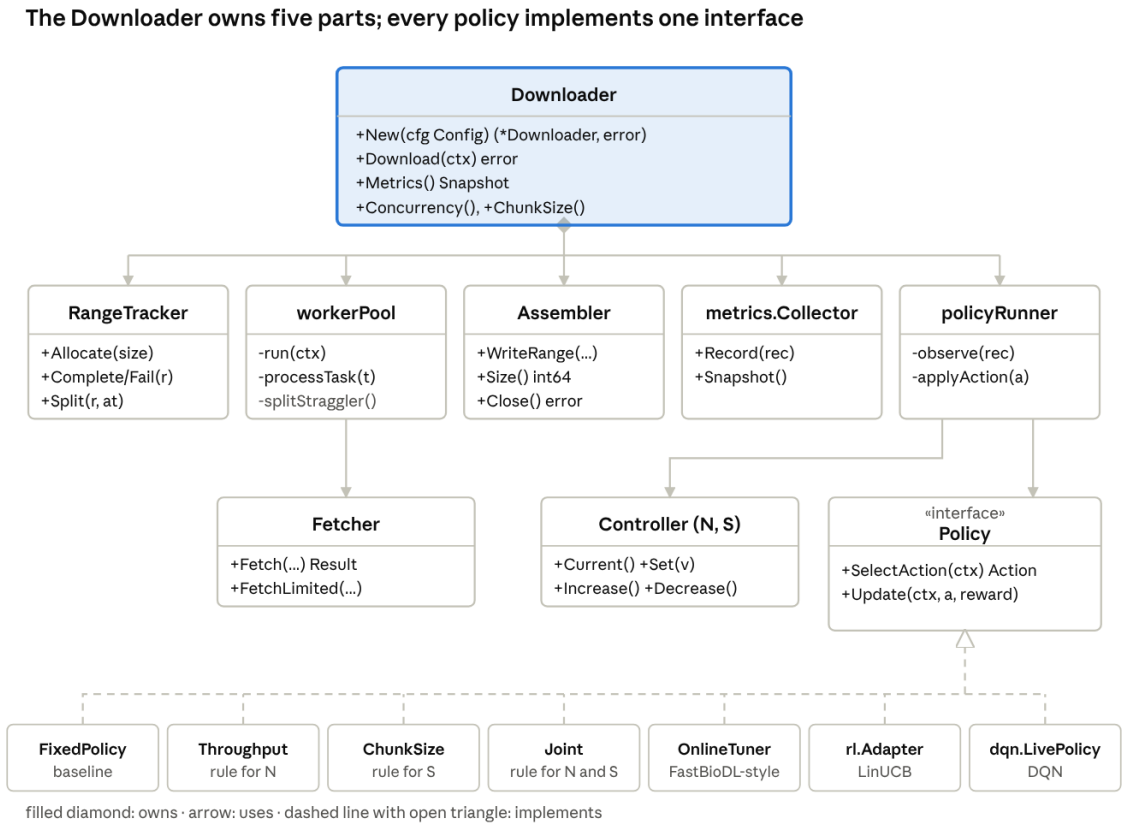


Figure 2.2: UML class diagram of the final implementation. Every policy, rule-based, learned or online-tuned, implements one interface.

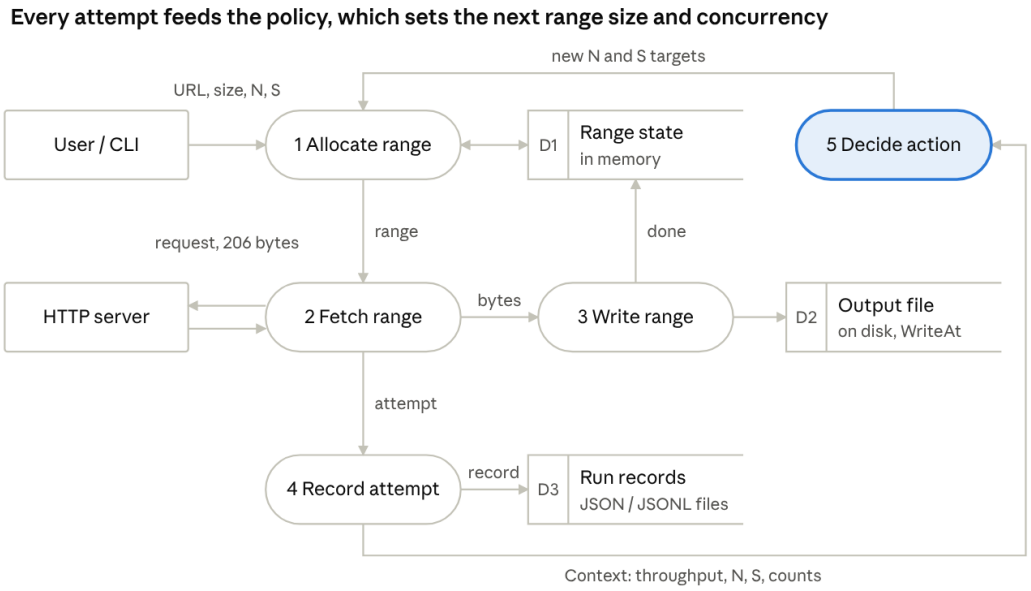


Figure 2.3: Level-1 data flow. Each attempt's result reaches the policy, which can change N and S for later allocations.

3. Complete Implementation

Table 3.1: Implemented modules

Module	Contents
`rangetracker`	Half-open range allocation, completion, failure, and splitting of in-flight ranges
`fetcher`	One HTTP/1.1 Range request per call; checks status, `Content-Range` and length; a shrinkable end for splitting
`assembler`	Writes ranges at their offsets with `WriteAt`
`downloader`	Worker pool, serialized policy runner, retries with unchanged bounds, opt-in straggler splitting
`concurrency`, `chunksize`	Bounded target controllers for N and S
`policy`	Policy interface; fixed, throughput, chunk-size, joint and online-tuner policies
`rl`	State, action and reward contracts; LinUCB (Gonum); DQN inference in Go
`metrics`	One record per attempt (bytes, duration, response latency, status); aggregates
`experiment`	Test server (pacing, cap, faults), netem control, trace replay, runners
`python/`	DQN training (PyTorch), simulator, analysis scripts

3.1 Key Algorithms

Download loop. Each worker allocates the next range at the current size S, fetches it with one Range request, records the attempt, and then either writes it or retries the same range (Figure 3.1).

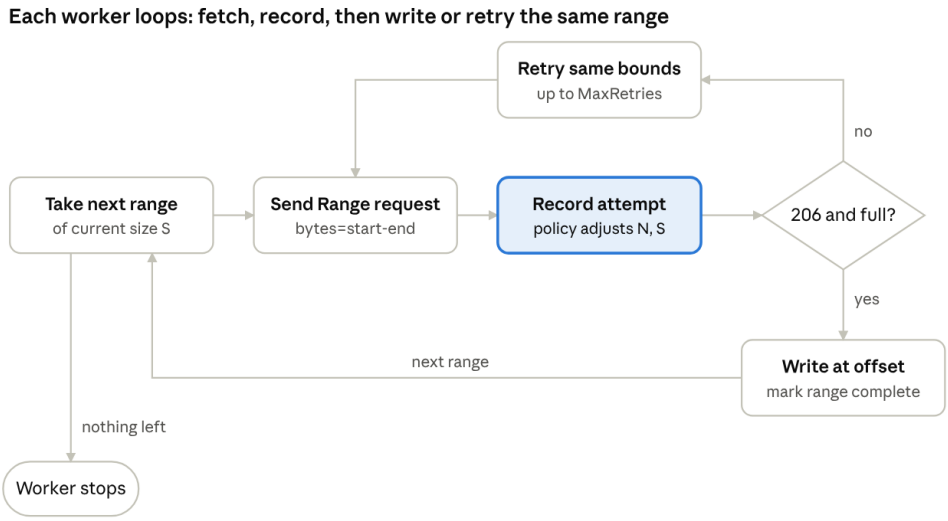


Figure 3.1: Worker algorithm.

Straggler splitting. When the tracker has nothing left to allocate, an idle worker picks the in-flight range with the most unreceived bytes, $r = e - (s + b)$, where b bytes have arrived. If $r \geq 2$ MiB (so each half is at least 1 MiB and a split is worth a new request), it moves that request's end to

$$m = s + b + \lfloor r/2 \rfloor \tag{3.1}$$

and fetches $[m, e)$ itself. Bytes already received stay with the original request.

Online concurrency tuner. Every $\Delta t = 2$ s the tuner measures aggregate throughput T, the bytes of all attempts completed in the window divided by Δt. Its score is $U = T/K^N$, where K is a per-connection penalty in the style of Falcon; we set $K = 1$, so the score is simply T, as in FastBioDL's throughput objective. If U is at least 5 % worse than the previous window, it reverses direction and halves its step. Otherwise it keeps moving N the same way. It starts at $N = 4$ with a step of 4, within $[1, 16]$. A move of several connections is sent as one action carrying a step count, which the runner applies as that many unit steps.

Causal drop controller. It tracks the running maximum of the windowed median per-request throughput, and switches once to a predeclared target (N, S) when throughput falls below half of that maximum. The target, 4×4 MiB, was fixed in advance on the original engine. The controller reaches it through ordinary increase and decrease actions, several steps at a time.

Workload selector. It starts at 16×1 MiB, watches per-request throughput for the first 20 samples, classifies the server as capped or not by a predeclared threshold, and then moves once to that class's predeclared target.

Learned policies. LinUCB is a contextual bandit over the five generic actions (none, $\pm N$, $\pm S$). Its context is 10 normalised features: N, S, per-request and aggregate throughput, request duration, bytes, and success, failure, retry and attempt counts. Its reward combines aggregate throughput and its change, minus penalties for latency, retries, failures and actions. The DQN agents are Double DQN networks trained in PyTorch on a simulator of the download process, with 13 features (adding remaining bytes and throughput trend, volatility and time-since-change) and reward $r_t = -\Delta t$, so the episode return is the negative completion time. Versions v2 and v3 acted on both N and S. From v4 on, the agent acts on N

only, for the reasons in Table 8.2. The trained network runs inside Go for live decisions.

4. Live Demonstration

The demonstration has three steps. The captures below are real terminal output from the project machine.

1. Download a public file from `cdn.kernel.org` and verify it against kernel.org's published checksum (Figure 4.1).
2. Show the benchmark of our downloader against aria2 on the paced test server (Figure 4.2).
3. Run the automated test suite (Figure 4.3).

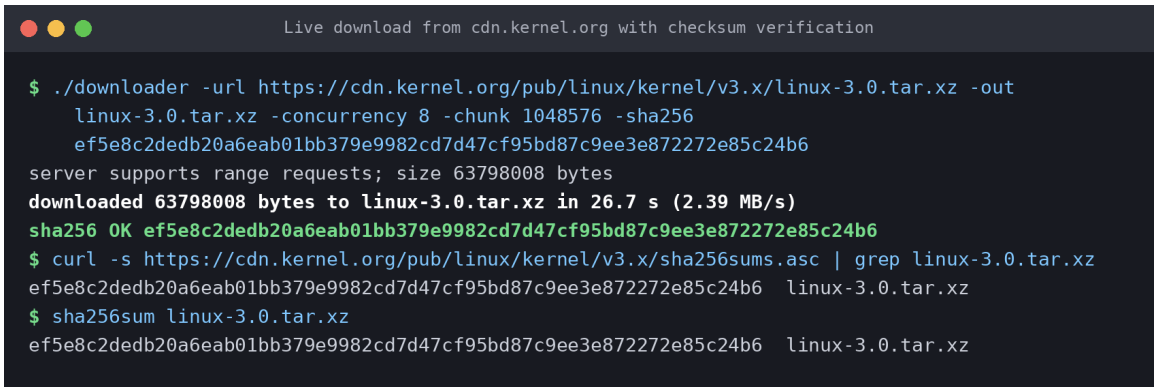


Figure 4.1: Live download of `linux-3.0.tar.xz` (63.8 MB) from cdn.kernel.org with 8 connections; the SHA-256 matches the published value.

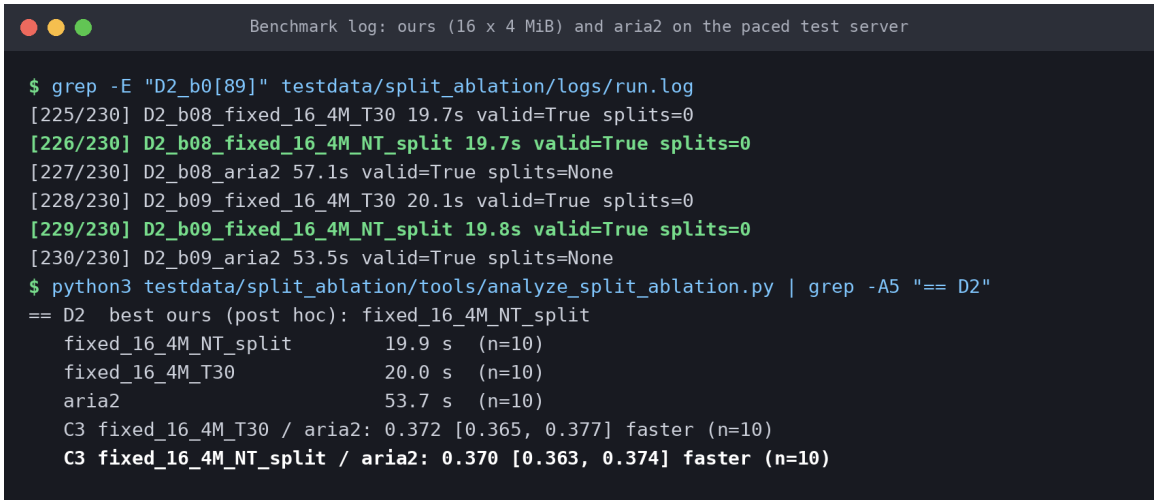


Figure 4.2: Benchmark log on the paced test server: ours about 20 s, aria2 about 54 s.

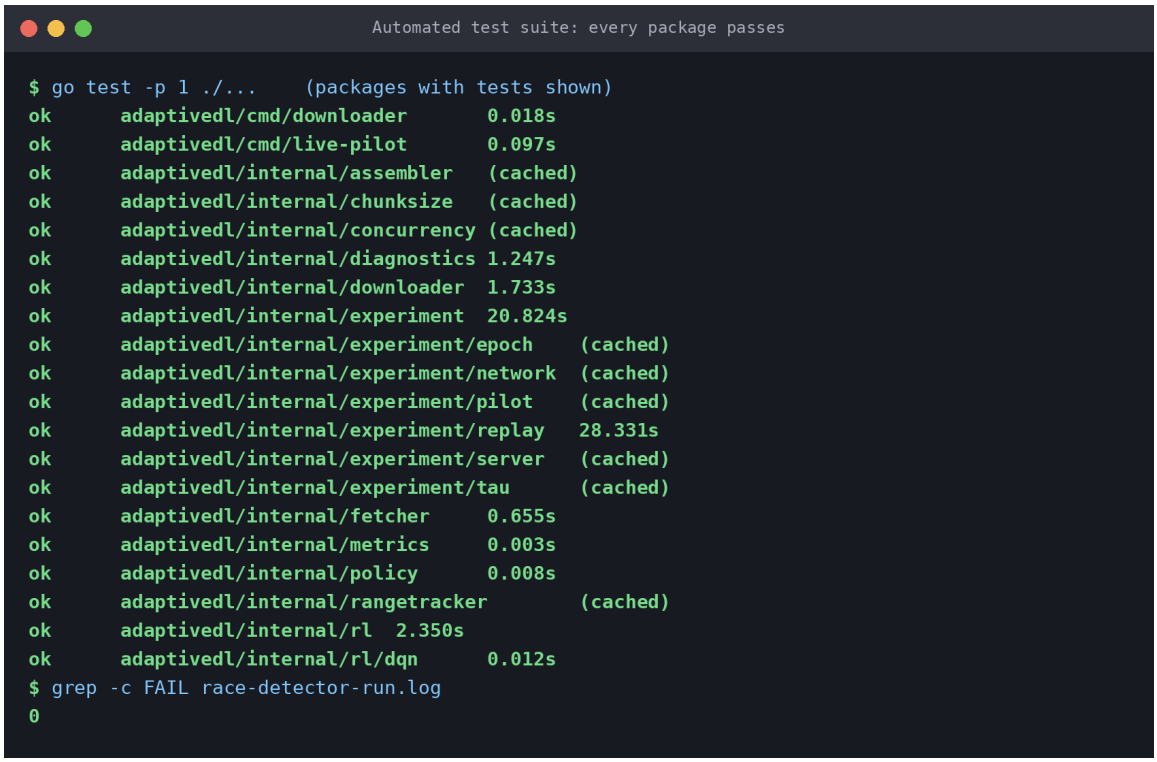


Figure 4.3: The automated Go test suite passing.

5. Test Results

The project has 549 automated Go test functions in 29 packages. They cover range allocation, reply validation, out-of-order writes, retries, concurrency changes, policy decisions, straggler splitting, the online tuner and the CLI probe. No test needs internet access.

Table 5.1: Verification summary

Check	Result
Unit and integration tests (`go test ./...`)	All pass
Race detector (`go test -race ./...`)	No data races
Repeated runs (`go test -count=5 ./...`)	Pass
Static checks (`go vet`, `gofmt`)	Clean
Paper claims (`tools/claims_check.py`)	144 of 144 numbers match the raw data
Experiment validity, engine ablation	230 of 230 runs valid and content-verified
Experiment validity, live-internet pilot	every run SHA-256 checked; invalid runs kept and reported

Table 5.2: Test cases for the final additions

No.	Test case	Expected result	Status
1	Slow first range, idle worker, splitting on	Worker takes the second half; file byte-identical	Pass
2	A split range then fails	Retry asks only for the shortened range	Pass
3	Splitting off (default)	No split happens	Pass
4	Tuner: throughput keeps rising	N keeps moving up by the current step	Pass
5	Tuner: a window 5 % or more worse	Direction reverses, step halves	Pass
6	Tuner at $N = 16$	Holds until a worse window	Pass
7	CLI probe against a server without Range support	Rejected before downloading	Pass
8	CLI with a wrong expected SHA-256	Download reported as failed	Pass
9	Server delays its response headers	Latency recorded, and at most the request duration	Pass

5.1 Reliability

Figure 5.1 shows the reliability problem the final evaluation uncovered. The original engine used a 30 s deadline over the whole request. On the capped server, large ranges ran past it and restarted from zero, so up to 112 % of the file was downloaded again, and the runs at 8 and 16 connections aborted. Every

adaptive arm in the ablation and the tuner study also had zero wasted bytes. In the engine ablation, all 230 runs were valid and had **zero** failed attempts, **zero** retries and **zero** wasted bytes. That includes its arms that kept the 30 s deadline, because they used 1–4 MiB ranges, which finish well inside it. The aborted runs in Figure 5.1 come from the earlier (N, S) grid on the capped server, not from the ablation.

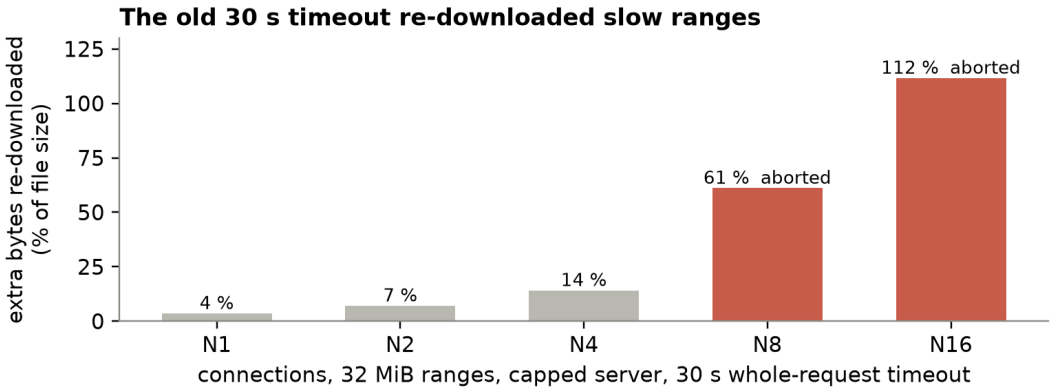


Figure 5.1: Extra bytes re-downloaded under the original 30 s whole-request timeout (capped server, 32 MiB ranges).

5.2 Goodput

Goodput is the useful data delivered per second: the file size divided by the completion time. Transfer efficiency is the useful bytes divided by all bytes transferred. On the fixed engine, efficiency was 100 % in every run. On the original engine with 32 MiB ranges it fell as low as 47 % (112 % extra bytes, Figure 5.1). Table 5.3 gives goodput for our best fixed setting and for aria2 in each ablation scenario.

Table 5.3: Goodput in MB/s (geometric mean of 10 runs; fixed engine)

Scenario	Our best fixed setting	Ours	aria2
Cap from start	16 × 32 MiB, splitting	7.87	7.83
Cap from 8 s	16 × 32 MiB, splitting	9.87	10.03
No cap	16 × 1 MiB	29.19	10.05
Paced (D2)	16 × 4 MiB	27.02	9.99

6. Performance Analysis

All comparisons are paired by block, with geometric-mean time ratios R and 95 % bootstrap intervals. $R < 1$ means the first option was faster. Section 6.1 gives the full configuration.

6.1 Experimental Setup

Table 6.1: Platform

Item	Configuration
Host (project machine)	AMD Ryzen 5 5500U laptop (6 cores, 12 threads); client and test server on the same machine
Operating system	Ubuntu 24.04 container on a Linux 7.x kernel
Software	Go 1.27.1; aria2 1.37.0; Python 3.14 with PyTorch 2.9 (CPU) and NumPy 2.5; Mahimahi for trace replay
Network emulation	`tc netem` on the loopback interface: 40 ms delay each way (80 ms round trip, measured 80.1 ms) and 0.4 % loss each way
Test file	512 MiB with deterministic content, so every output can be verified byte for byte
aria2 settings	`-x16 -s16 -k1M` (16 connections, 1 MiB minimum piece), the same in every run

Table 6.2: Server scenarios

Scenario	Server behaviour
Paced (D2)	Each response is sent at 640 KiB/s, 160 KiB/s from 2 s, and 2,500 KiB/s from 4 s
No cap (HEALTHY)	640 KiB/s per response, rising to 2,500 KiB/s from 1 s
Cap from start (CAP0)	As No cap, plus a soft concurrency cap from the start
Cap from 8 s (CAP8)	As No cap, with the cap switching on at 8 s
Stock NGINX	NGINX 1.24 with `limit_rate` and `limit_rate_after`, per response

Paced test server. The main test server is part of this project. It serves the 512 MiB file, honours Range requests and follows three rules:

1. Each response is sent at the server's rate at the moment its request arrives, and keeps that rate until it finishes.
2. A schedule changes the rate at fixed times; a change affects only requests that start afterwards.
3. Without the cap, rates are per response with no shared limit, so more connections add bandwidth.

Because the rules are explicit, each result can be traced to them. Figure 6.1 shows the reference schedule, D2.

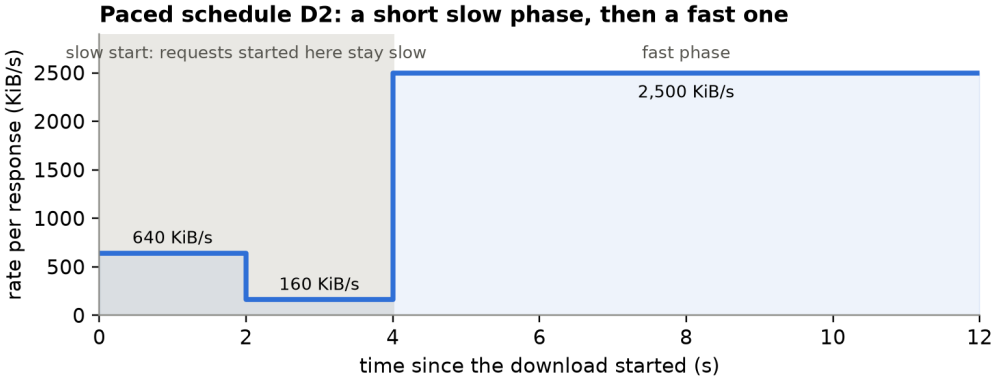


Figure 6.1: Reference pacing schedule D2. A request that starts in the first 4 s keeps its slow rate until it finishes.

Under the cap, each request's rate is fixed when it starts. With k requests active at that moment,

$$\text{rate} = \min\left(\text{base},\ \frac{C}{k\,(1+\gamma(k-1))}\right), \qquad C = 8\ \text{MiB/s},\ \gamma = 0.05, \tag{6.1}$$

so the total falls slightly as more connections are opened. The model imitates servers that share a fixed budget across a client's connections and penalise many parallel ones. C and γ were chosen during design, from a search over candidate server models, so that the cap changes which setting is best. Because a request keeps its starting rate, requests started before the cap switches on keep their higher rate, which is why many early, long requests can still win.

Run design. Each experiment runs in blocks. Every block holds one run of each arm in a random order fixed by a seed, and every run uses a fresh server, output file and netem setup. The engine ablation used 10 blocks per scenario, and the online-tuner study used 8. A run is valid only if it exits cleanly, has no error and its output matches the source byte for byte. Invalid runs are kept and reported, never dropped. Each protocol, with its comparisons and decision rules, was written down before its first run, and later amendments are dated and disclosed.

Real-network studies.

- *Live internet:* `linux-3.0.tar.xz` (63.8 MB), downloaded from four public mirrors: the kernel.org CDN and edge mirror, RIKEN (Japan) and TUNA (Tsinghua University, China). The client was on a residential broadband link. Eight fixed (N,S) settings ($N \in \{1,4,8,16\}$, $S \in \{1,4\}$ MiB), aria2 and curl were each run twice per condition, and every run was checked against the published SHA-256.
- *Recorded cellular traces:* five Mahimahi traces (Verizon EVDO and LTE, AT&T LTE, T-Mobile LTE and UMTS), replayed through `mm-link`.

Statistics. Each result is the geometric mean of per-block time ratios, with a 95 % percentile bootstrap interval over blocks from 10,000 resamples. A difference counts as real only when the interval excludes 1.

6.2 Engine Ablation (230 runs)

The ablation tested why aria2 had been faster on the capped server. Each scenario ran 10 blocks of its arms (Table 6.3), with 230 runs in all. Figure 6.2 shows the result.

Table 6.3: Engine-ablation arms. "T30": original 30 s whole-request deadline, no splitting. "fixed": no deadline, splitting on.

Scenario	Arms	Runs
Cap from 8 s	aria2; 16×1 MiB T30; 16×32 MiB without deadline, with and without splitting; 16×1 MiB, 4×4 MiB fixed; causal controller T30 and fixed	80
Cap from start	aria2; 16×1 MiB T30; 16×32 MiB with and without splitting; 16×1 MiB, 4×4 MiB fixed; selector fixed	70
No cap	aria2; 16×1 MiB T30 and fixed; 16×32 MiB fixed; selector fixed	50
Paced (D2)	aria2; 16×4 MiB T30 and fixed	30

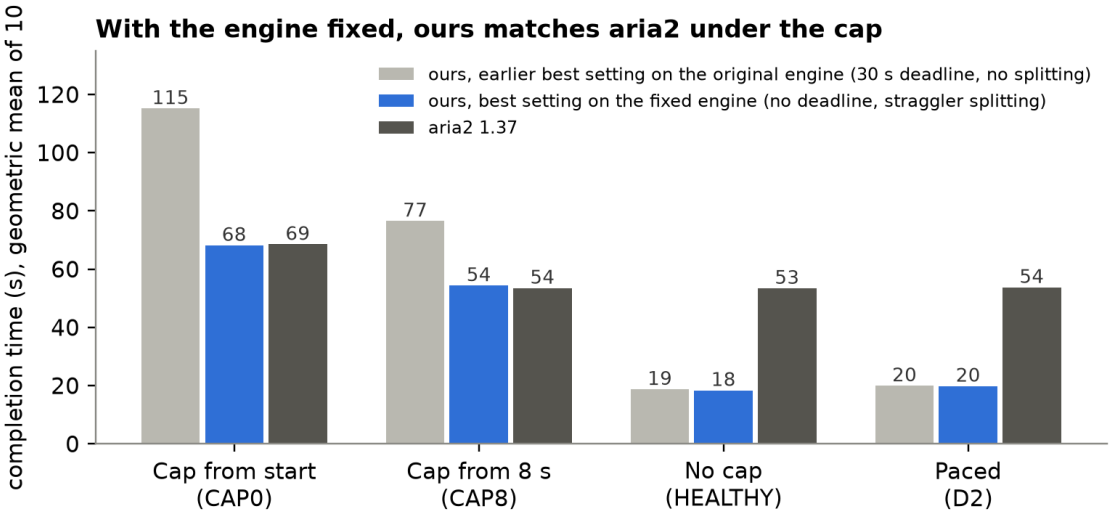


Figure 6.2: Completion time before and after the engine fix, and aria2, in four scenarios (10 blocks each).

- **Straggler splitting was the main cause.** With the cap on from the start, the same 16 × 32 MiB setting took 40 % less time with splitting than without ($R = 0.599$ [0.598, 0.601]). It then matched aria2: 68.2 s against 68.5 s, $R = 0.995$ [0.992, 0.999], a difference that is statistically detectable but practically negligible.
- **Large ranges need the timeout removed.** With the cap switching on at 8 s, 16 × 32 MiB without the timeout was 29 % faster than the earlier best fixed setting, 16 × 1 MiB ($R = 0.711$). Our best setting there was within 1.6 % of aria2 (54.4 s against 53.5 s, $R = 1.016$ [1.006, 1.029]), a small but real gap.

- **No harm elsewhere.** Without a cap, and on the paced server, the same setting on the fixed and the original engine was not distinguishable ($R = 0.981$ and 0.995).

6.3 Does Adaptation Help?

In an earlier predeclared comparison, a causal controller was 23.4 % faster than the best of three fixed settings when the cap switched on at 8 s. That result replicated here ($R = 0.767$). Most of the gain came from the N move: an N-only version reached 0.808 of the fixed setting's time, while an S-only version was slower than not adapting at all (1.070). But those three fixed settings used small ranges (1–4 MiB) on the original engine. Against the best fixed setting on the fixed engine (16 × 32 MiB), the causal controller was 7 % *slower*, and the workload selector was 23 % slower (Figure 6.3).

What "best fixed setting" means. The best fixed setting in each comparison is chosen after the fact, which an adaptive method cannot do. Two results show when this matters. On the live mirrors, one setting chosen for all conditions (8 × 1 MiB) was within 3.7 % of the per-condition best, so knowing the conditions in advance was worth little. Between servers with and without a cap, no single setting works: 16 × 1 MiB is best without the cap (18.4 s) but slow with it (115.2 s). There, a one-time choice made from early measurements did as well as knowing the workload in advance: the workload selector matched the per-workload best ($R = 0.999$ [0.969, 1.029], measured on the original engine).

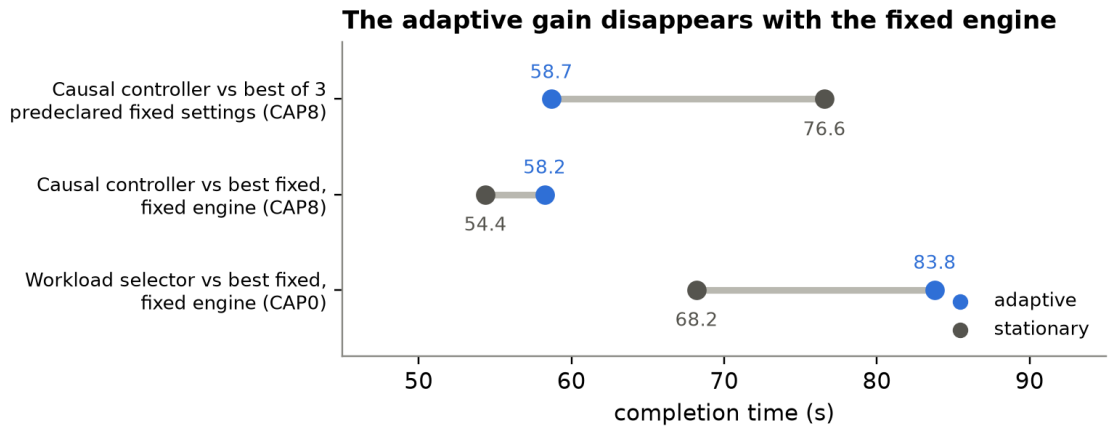


Figure 6.3: Adaptive controllers against the best stationary setting.

6.4 Adaptation Speed

The decision logs record when each controller acts. Table 6.4 reports how quickly each adaptive controller reacted.

Table 6.4: Adaptation speed: median seconds, with the range over runs

Controller	Scenario	Measured	Seconds	Runs
Causal drop controller	Cap switches on at 8 s	first N cut after the cap	1.5 [1.5, 1.7]	10
Online tuner	Cap switches on at 8 s	first N cut after the cap	2.1 [2.1, 2.3]	8
Workload selector	Cap from the start	first move (classification)	4.0 [4.0, 4.2]	10
Online tuner	No cap	climb from $N = 4$ to $N = 16$	8.1 [8.0, 8.2]	8

In every case the controller reacted within a few seconds, so slow reaction is not why adaptation failed to pay off. The limit is what the reaction can gain: the best fixed setting was already close to optimal.

6.5 Online Concurrency Tuner

The online tuner (Section 3.1) was compared with two fixed settings on the same engine and with aria2. The protocol was written down before the first run: 2 scenarios × 4 arms × 8 blocks, with 0 invalid runs.

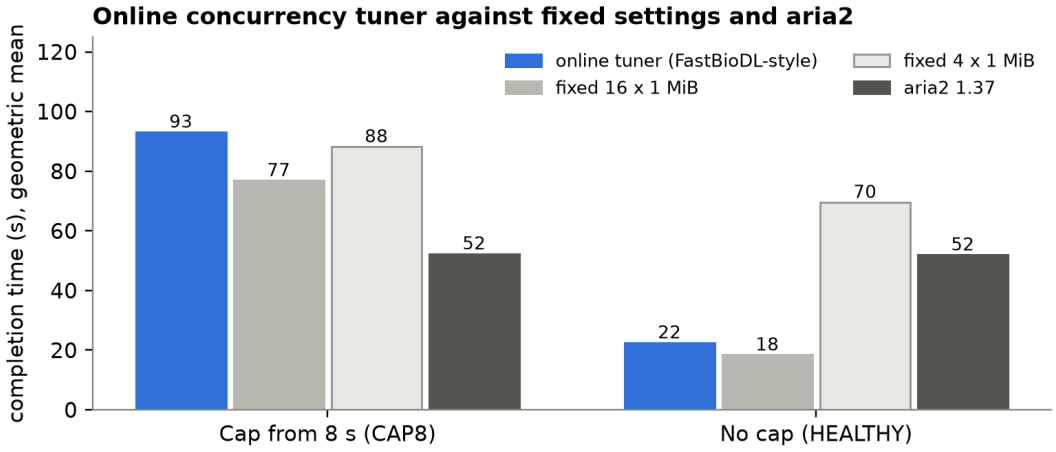


Figure 6.4: Online concurrency tuner against fixed settings and aria2 (8 blocks per scenario).

Table 6.5: Online tuner results; ratios are tuner / other

Scenario	Tuner (s)	Best fixed (s)	vs best fixed	vs aria2	Final N
Cap from 8 s	93.1	16 × 1 MiB, 76.8	1.212 [1.180, 1.248] (slower)	1.781 [1.718, 1.844] (slower)	12.5
No cap	22.5	16 × 1 MiB, 18.4	1.220 [1.203, 1.232] (slower)	0.432 [0.416, 0.452] (faster)	16

The tuner was not faster than the best fixed setting in either scenario. So an online tuner of the kind used in prior work also did not beat a well-chosen fixed setting on this platform. The tuner adapts concurrency only and used 1 MiB ranges, so it is compared with fixed settings at the same range size. With the cap from 8 s, the best fixed setting in the engine ablation, 16 × 32 MiB with splitting (54.4 s), was faster still. The tuner reacted to the cap within about 2 s (Table 6.4), but with 1 MiB ranges almost

every request starts after the cap, so all of them run at the capped rate. Large ranges started before the cap keep the higher rate, which is why 16×32 MiB does better.

6.6 Other Searches for an Adaptation Opportunity

Besides the main experiments, we looked for conditions where adapting N or S during a download could pay off. Table 6.6 lists each study and its result.

Table 6.6: Studies that looked for a benefit from adaptation ("Testbed": local server and emulated network)

Study	Type	Result
Rule-based adaptive controllers (N-only, S-only, joint) on the paced server	Testbed, 80 runs	All slower than the best fixed setting; the joint controller took $2.32\times$ as long ($R = 2.316$ [2.302, 2.329])
Workload selector over six server workloads	Testbed, 240 runs	Matched the best fixed setting for each workload ($R = 0.999$ [0.969, 1.029]); a gain over one family-wide setting was not established
Shared bottleneck with background TCP traffic (`iperf3`)	Testbed, 45 runs	No state-dependent best N; under heavy contention more connections always won
Per-request server throttling	Testbed	No adaptive-concurrency opportunity
Range-size sweep across server phases	Testbed	No reversal in which range size is best
Adaptive S, adaptive N, joint pacing and correlated-loss mechanisms	Simulation	No exploitable mechanism found

6.7 Real Networks

Live internet. Across 16 conditions on four public mirrors, one fixed setting (8×1 MiB) was within 3.7 % [1.0, 4.8] of the best setting chosen separately for each condition. It was within 5 % in 11 of 16 conditions, where a condition is one mirror in one of four measurement sessions (Figure 6.5). Within these eight settings, choosing per condition instead of one global setting gained at most 3.7 % on average. Parallel connections clearly mattered: a single curl connection took 1.03 to 6.5 times as long as the best setting, depending on the mirror. But once several connections were used, the exact choice mattered little, partly because the client's access link (about 2.4 MB/s) became the limit.

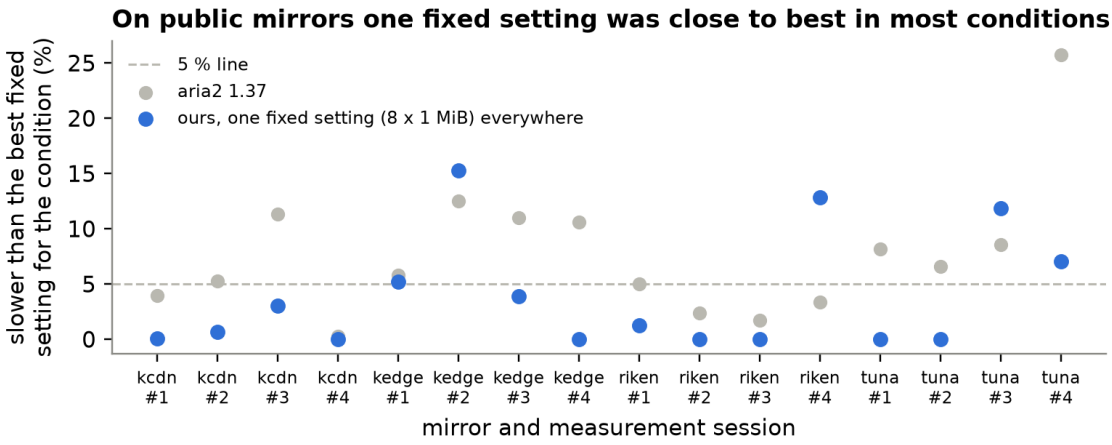


Figure 6.5: Live-internet pilot: how much slower one global fixed setting and aria2 were than the per-condition best.

Recorded cellular traces. Replaying five recorded traces, a clairvoyant oracle found no exploitable concurrency headroom: the one-sided 95 % lower bound was −0.26 %, against a predeclared 10 % bar.

Learned policies. Table 6.7 summarises the learned-policy evaluations.

Table 6.7: Learned-policy evaluations (ratio > 1: slower than the comparison)

Policy	Evaluation	Result
LinUCB (joint N, S)	300 runs, 5 scenarios, against fixed and rule-based modes	Locked onto one action after warm-up; no ranking possible
DQN v2 (joint N, S)	240 pairs against aria2 (4 MiB files, aria2 with 4 connections)	1.757 [1.672, 1.847]
DQN v5 (N only)	240 pairs against aria2, 16 MiB files	2.322 [2.230, 2.417]
DQN v6 (N only, 1–16)	150 pairs against aria2, 16 MiB files	1.456 [1.305, 1.624]
DQN v6, longer files	16/128/512 MiB against aria2	1.81 / 1.06 / 0.91 (512 MiB interval includes 1)

LinUCB locked onto a single action after warm-up. The DQN agents (v2, v5, v6) were slower than aria2 overall in the head-to-heads we ran. Those comparisons used small files (4–16 MiB), and v6 started from one connection while aria2 opened 16 at once, so they test the agents' start-up behaviour more than their steady-state control. A follow-up with longer downloads (16, 128 and 512 MiB) gave pooled DQN/aria2 ratios of 1.81, 1.06 and 0.91. The 512 MiB value has an interval that includes 1 and is driven by one of three scenarios; in the other two, DQN fell further behind as files grew. The head-to-heads compared DQN with aria2. In v5's live tests, the fixed 4-connection setting was fastest in every tested condition.

6.8 Limitations

- **Constructed server.** The capped and paced servers are models we built. Their key rule, that a request keeps the rate it started with, drives both the 62.4 % result and aria2's behaviour under the

cap. A server that re-evaluates rates during a response could change both results. The stock-NGINX check exercised per-response limits only, not a shared cap.

- **One machine, emulated network.** Every controlled result ran on one machine, over loopback with `netem`, with one aria2 version. Real paths appear only in the live pilot and the trace replay.
- **Live pilot scope.** The pilot used one client, one access link, four mirrors and two runs per setting, and ran fixed settings only. It bounds what adaptation could gain on that link; it is not a survey of networks.
- **One configuration per controller.** Each adaptive controller and the online tuner ran with one configuration, fixed before its runs. Other parameters might do better, though the small headroom found in every study limits how much.
- **Sample sizes and statistics.** The main comparisons use 8–30 paired blocks. The pacing sweep and the NGINX check (5 pairs each) are screens, not confirmations. Because the testbed is nearly deterministic, intervals are narrow and reflect testbed noise, not real-world variability. No correction was applied across the many comparisons.
- **No request timeout in the fixed engine.** Removing the whole-request deadline means a connection that stalls completely would wait until the run is stopped. A stall-based timeout is future work.
- **Fairness.** Opening 16 connections takes a larger share of a shared link from other flows, and places more load on public mirrors. We measured speed, not fairness.

7. Comparison with Existing System

The existing system is aria2 1.37, run with the same flags throughout (`-x16 -s16 -k1M`). We chose aria2 because it is the standard open-source segmented downloader on Linux. curl was run as a single-connection reference in the live pilot, and IDM runs only on Windows.

Table 7.1: Ours (best fixed setting, fixed engine) against aria2

Scenario	**Ratio R**	**Verdict**
Paced server (D2), 16 × 4 MiB	0.370 [0.363, 0.374]	Ours faster
No cap, 16 × 1 MiB	0.344 [0.341, 0.347]	Ours faster
Stock NGINX with `limit_rate` (screen, 5 pairs each)	0.597–0.863	Ours faster, 10 of 10 pairs
Cap from start, 16 × 32 MiB with splitting	0.995 [0.992, 0.999]	Level in practice (ours 0.5 % faster)
Cap from 8 s, 16 × 32 MiB	1.016 [1.006, 1.029]	aria2 1.6 % faster
Live public mirrors (16 conditions)	aria2 1.00–1.26 × per-condition best	aria2 within 0–26 %; ours (8×1 MiB) within 0–15 %

7.1 What the Comparison Shows

These results do not show a general advantage over aria2. Our large wins come from servers that set each response's rate when it starts. There, our smaller ranges keep starting new requests after the server speeds up, while aria2's 32 MiB first pieces stay at the slow starting rate. Not all of the gap is explained: with a constant fast rate and no rate change, ours was still faster (15.4 s against 27.1 s), and with aria2-like piece sizes ours remained 14 % faster. We did not isolate that residual. When the conditions remove the timing effect, the two are level or close: at a constant slow rate, under the cap once our engine splits stragglers too (level from the start, aria2 1.6 % faster when the cap starts at 8 s), and on public mirrors.

Functional differences. Unlike aria2, our downloader can change N and S during a download through a policy, records every request attempt, and verifies each reply's `Content-Range`. aria2 supports many more protocols and features (BitTorrent, Metalink, multiple sources), which are outside this project's scope. IDM was not evaluated because it runs only on Windows.

8. Achievement of Objectives

Table 8.1: Review 1 objectives and outcomes

Objective (Review 1)	**Status**	**Outcome**
Correct segmented downloader, one range per request	Achieved	Byte-identical output; probe and SHA-256 check in the CLI
Controlled test server with throttling and faults	Achieved	Pacing, concurrency cap, fault injection
Network monitor over rolling windows	Achieved	A rolling window over recent attempts gives per-request and aggregate throughput, response latency (time to first byte), failures and retries
Policy interface: fixed, heuristic, LinUCB, RL	Achieved	One interface runs all of them
Joint adaptation of N and S	Achieved	Implemented and evaluated in rule-based, causal, LinUCB and DQN (v2, v3) controllers; joint learning did not beat fixed settings, so the final DQN adapts N only
Evaluate under stable, variable, lossy, throttled and background-traffic conditions	Achieved	Server rate schedules for variable bandwidth; netem delay and loss (one setting, 80 ms and 0.4 %); recorded cellular traces and the live internet for real variation; capped servers; background TCP traffic
Compare policies on time, goodput, reliability and adaptation	Achieved	Completion time, goodput, reliability (failures, retries, wasted bytes) and adaptation speed (Section 6.4)
N-only, S-only and joint ablation	Achieved	With the cap from 8 s, N-only 0.808 and joint 0.766 of the fixed time; S-only 1.070 (Section 6.3)

8.1 Deviations from the Review 1 Plan

Table 8.2: Deviations and their reasons

Deviation	Reason
Title and framing: from building an RL download manager to evaluating when adaptation helps	The early results showed well-chosen fixed settings close to optimal, so the central question became when adaptation pays off. The system, scope and policies stayed as planned.
Final DQN learns N only, with S fixed at 1 MiB	DQN v2 fragmented ranges into 32–128 KiB pieces. v3's simulator rewarded larger ranges that the real server did not reward. Range size was therefore removed from the learned action (v4 onward).
No PPO or hybrid heuristic-plus-RL agent	After LinUCB locked onto one action, Double DQN was chosen because it handles credit over time; it was carried through to live evaluation. A PPO hybrid is listed as future work.
Decisions after each completed request, not every fixed interval T	The live runner decides once per completed attempt. The DQN simulator was aligned to the same cadence after a mismatch was found in v5.
In-flight ranges were never split (Review 1 contrasted this with IDM)	Kept for correctness and simple accounting. The final ablation showed this was the main reason aria2 led under the cap, so splitting was added as an opt-in option.
No resume from disk	Outside the evaluated scope; every experiment needs a fresh download.
Cobra, slog, gRPC and `x/time/rate` not used	The standard library was enough. The DQN runs inside Go from an exported model, so no RPC was needed. The server needed per-response pacing and a concurrency-dependent cap, so it uses its own rate model.
Prior-work online optimizer evaluated as a re-implementation	FastBioDL's own code was not used; its online-tuning idea was re-implemented as a project policy and evaluated under a frozen protocol (Section 6.5).

8.2 Use of Prior Work

Table 8.3: Techniques from the Review 1 literature survey and how this project used them

Prior work	Used?	How, or why not
Liu et al. 2018, deep RL for transfer parameters	Yes, adapted	Double DQN agents (v2–v6). The final agent learns *N* only, for the reasons in Table 8.2.
FastBioDL 2025, online concurrency optimization	Yes, re-implemented	The online tuner in Section 6.5: the same idea, written from scratch, run under a frozen protocol.
Swargo and Arifuzzaman 2025, heuristics plus PPO	No	One learning algorithm (DQN) was carried through to live evaluation; a PPO hybrid is future work.
IDM and aria2, dynamic segmentation	Yes, opt-in	Straggler splitting; the ablation showed it is what closes the gap to aria2 under the cap.
DeLoad 2025, learned range sizing	No	It targets short-video preloading with a watch-time objective, which has no counterpart in a single-file download.
MDTP 2025, multi-source chunk allocation	No	Multiple sources were outside the single-server scope fixed in Review 1.
BOLA 2016, buffer-based adaptation	Analogy only	Cited as an example of decisions made without throughput prediction.

9. Innovation and Real-World Application

Adapting connections and range size is an established idea. FastBioDL [3], Falcon [10] and direct-search tuning [11] adjust the number of streams online, Liu et al. [1] applied deep RL to transfer parameters, and download managers such as aria2 [8] split slow segments. This project does not claim to have invented these. Its contribution is a measured answer to a practical question: *when does adapting N and S during a download actually pay off?*

- **A mechanism-first test platform.** The test server makes its rate rules explicit, so each result can be traced to a cause. That is how the aria2 gap was traced to request timing and engine details rather than control.
- **Engine before policy.** On our server models, two engine details, the timeout semantics and straggler splitting, mattered more than any adaptive policy. Whether this holds on real throttling servers is untested.
- **Evidence discipline.** Each protocol was written down before its first run, every run is kept, and every reported number is recomputed from raw data.

Applications.

- *Download managers and software updaters:* on the mirrors we measured, one fixed setting such as 8×1 MiB was close to best. Straggler splitting, and a timeout that does not cut off slow but progressing requests, mattered more than adaptive control.
- *Servers that throttle by connection count:* on our capped-server model, large ranges with straggler splitting did best; this remains to be checked on real servers.
- *Research and teaching:* the platform and raw data can be reused to test other downloaders and controllers.

10. Future Enhancement

- **Joint learning on harder environments.** Joint learned control was built and evaluated here, but our servers offered little for it to exploit. A natural next step is to test it where the best (N, S) changes often and by a lot, such as servers with shifting per-connection limits.
- **Broader live study.** More networks, client locations and mirrors, including mobile networks.
- **HTTP/2 and multiple sources.** Download one file from several mirrors at once, as MDTP does.
- **Defaults.** After validation on real servers, make straggler splitting the default and add a stall-based timeout in place of the removed whole-request deadline.
- **More baselines.** Add a hybrid heuristic-plus-PPO agent and a Windows test of IDM.

11. Outcome

A research paper, *Request granularity and concurrency adaptation for HTTP range downloads under controlled server-side pacing: a mechanism-first evaluation*, has been drafted. All 144 of its numerical claims are verified against the raw data by an automated checker.

Table 11.1: Publication plan

Item	Status
Paper draft	Complete; final revision pending
Target	A networking workshop or a journal that reviews for soundness (for example IEEE Access)
Data and code	Repository with raw runs, frozen protocols and analysis scripts, to be archived with a DOI

References

[1] Z. Liu, R. Kettimuthu, I. Foster and P. H. Beckman, "Toward a smart data transfer node," *Future Generation Computer Systems*, vol. 89, pp. 10–18, 2018.

[2] R. M. Swargo and M. Arifuzzaman, "Elastic data transfer optimization with hybrid reinforcement learning," arXiv:2511.06159, 2025.

[3] R. M. Swargo, E. Arslan and M. Arifuzzaman, "Adaptive parallel downloader for large genomic datasets," arXiv:2508.05511, 2025.

[4] T. Liu et al., "DeLoad: Demand-driven short-video preloading with scalable watch-time estimation," arXiv:2510.18459, 2025.

[5] S. Abdollah, C. Partridge and S. Shannigrahi, "MDTP: An adaptive multi-source data transfer protocol," arXiv:2505.09597, 2025.

[6] K. Spiteri, R. Urgaonkar and R. K. Sitaraman, "BOLA: Near-optimal bitrate adaptation for online videos," arXiv:1601.06748, 2016.

[7] R. Fielding, M. Nottingham and J. Reschke, "HTTP Semantics," RFC 9110, IETF, 2022.

[8] aria2 project, "aria2: The ultra fast download utility," version 1.37. `https://aria2.github.io/`

[9] R. Netravali et al., "Mahimahi: Accurate record-and-replay for HTTP," USENIX ATC, 2015.

[10] M. Arifuzzaman, B. Bockelman, J. Basney and E. Arslan, "Falcon: Fair and efficient online file transfer optimization," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 8, pp. 2265–2278, 2023.

[11] P. Balaprakash, V. Morozov, R. Kettimuthu, K. Kumaran and I. Foster, "Improving data transfer throughput with direct search optimization," in *Proc. ICPP*, 2016, pp. 248–257.